

Actividad #1: Estudio de Caso - Sistema de Gestión de Incidentes (Help Desk)

Unidad 1: Desarrollo de software - Introducción y paradigmas de diseño

Arquitectura lógica, UML, patrones de diseño y Gitflow

Estudiante: _____

Repositorio GitHub: <https://github.com/USUARIO/REPOSITORIO-HELPDESK>

Fecha de entrega: 16 de mayo de 2026

1. Introducción

El presente documento propone la arquitectura lógica de un Sistema de Gestión de Incidentes tipo Help Desk. La solución permite registrar solicitudes técnicas, clasificar y priorizar incidentes, asignarlos a técnicos, dar seguimiento al estado, notificar cambios y generar reportes de gestión. El diseño busca escalabilidad, mantenibilidad y trazabilidad mediante UML, patrones de diseño y control de versiones con Gitflow.

2. Alcance del sistema

El sistema está orientado a organizaciones que requieren administrar tickets de soporte interno o externo. Cubre desde el registro del incidente hasta el cierre y evaluación básica del servicio. No se incluye la implementación completa del código fuente, pero sí la arquitectura lógica, los modelos principales y la estrategia profesional de versionamiento.

3. Actores del sistema

Actor	Descripción	Responsabilidades
Usuario / Solicitante	Persona que reporta un problema técnico.	Crear tickets, consultar estado, agregar evidencias y validar solución.
Técnico de soporte	Agente encargado de atender incidentes.	Diagnosticar, comentar avances, resolver, escalar o cerrar tickets.
Coordinador Help Desk	Responsable de la operación del equipo.	Asignar técnicos, priorizar casos, revisar SLA y reportes.
Administrador	Gestiona la configuración general.	Administrar usuarios, roles, categorías, permisos y parámetros del sistema.
Servicio de correo	Sistema externo de mensajería.	Recibir notificaciones automáticas de cambios en tickets.

4. Requerimientos funcionales

Código	Requerimiento
RF-01	El sistema debe permitir el registro de incidentes con título, descripción, categoría, prioridad y evidencias.
RF-02	El usuario debe consultar el estado y el historial de sus tickets.
RF-03	El coordinador debe asignar tickets a técnicos disponibles.
RF-04	El técnico debe registrar diagnóstico, comentarios, solución y evidencia de atención.
RF-05	El sistema debe permitir escalar un incidente a otro nivel de soporte.
RF-06	El sistema debe notificar por correo los cambios de estado, asignación y cierre.
RF-07	El administrador debe gestionar usuarios, roles, categorías y SLA.
RF-08	El sistema debe generar reportes de tickets por estado, prioridad, técnico y tiempo de respuesta.

5. Requerimientos no funcionales

Categoría	Requerimiento
Seguridad	Autenticación, autorización por roles y protección de datos sensibles.
Disponibilidad	Acceso web con disponibilidad objetivo del 99% en horario laboral.
Escalabilidad	Arquitectura modular que permita agregar canales como chat o WhatsApp.

Mantenibilidad	Código organizado por capas, aplicación de patrones y pruebas unitarias.
Trazabilidad	Historial completo de cambios en cada ticket y control de versiones en GitHub.
Usabilidad	Interfaz sencilla para registrar y consultar tickets sin capacitación extensa.

6. Arquitectura lógica propuesta

Se propone una arquitectura por capas para separar responsabilidades y facilitar pruebas, mantenimiento y evolución del sistema:

Capa	Responsabilidad	Componentes sugeridos
Presentación	Interacción con usuarios y validaciones básicas.	Pantallas web, formularios, panel de tickets, dashboards.
Aplicación / Servicios	Orquestación de casos de uso.	TicketService, UserService, ReportService, NotificationService.
Dominio	Reglas de negocio principales.	Ticket, Usuario, Categoría, Prioridad, Estado, SLA.
Infraestructura	Persistencia e integración externa.	Repositorios, base de datos, adaptador de correo, APIs externas.

7. Diagrama de casos de uso UML



Figura 1. Casos de uso principales del Sistema Help Desk.

8. Diagrama de clases UML

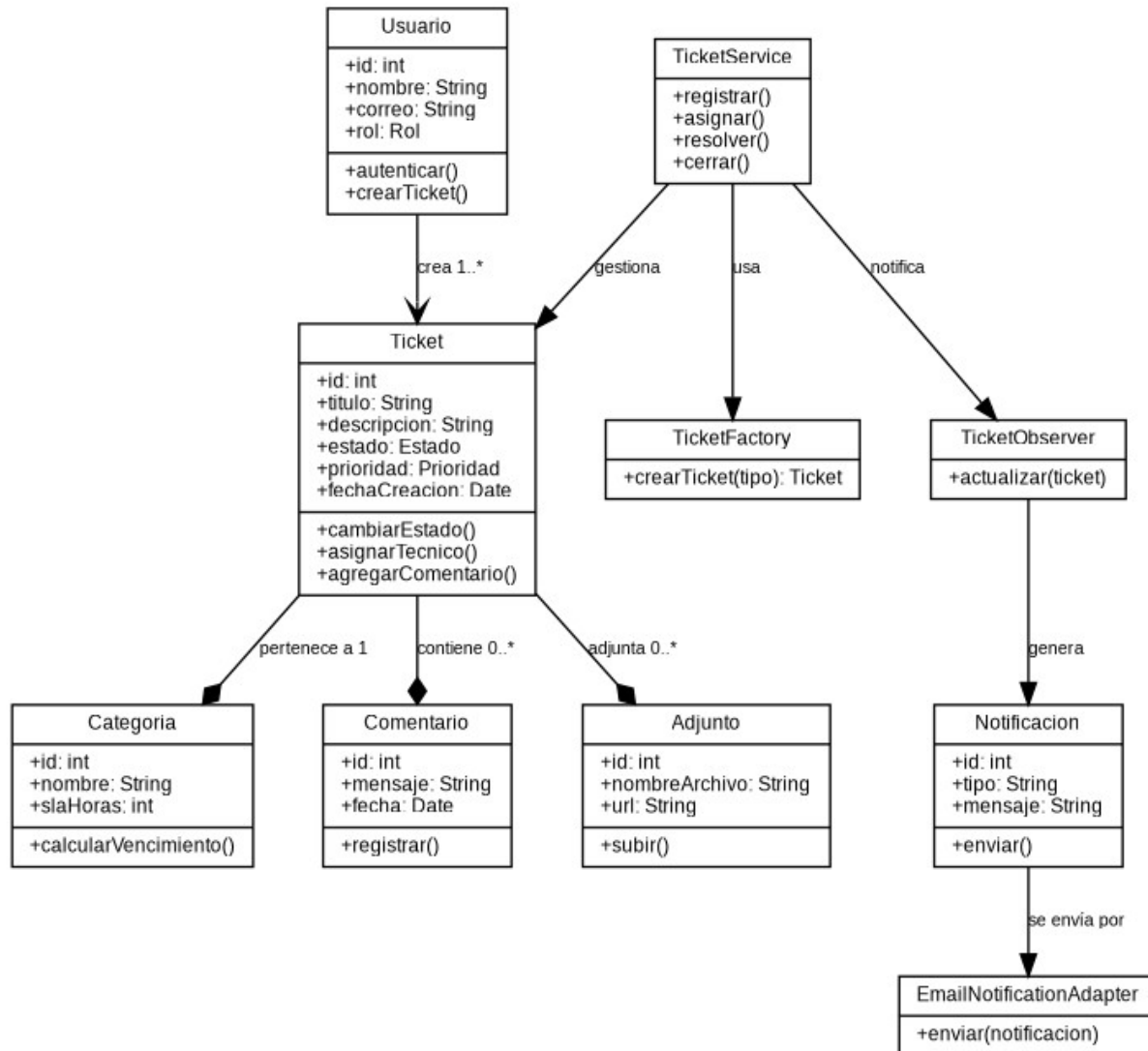


Figura 2. Modelo de clases lógico con entidades, servicios y patrones asociados.

9. Patrones de diseño implementados

Patrón	Tipo	Aplicación en el sistema	Beneficio
Factory Method	Creacional	TicketFactory crea tickets según tipo: incidente, requerimiento, problema o cambio.	Centraliza la creación y permite extender nuevos tipos sin modificar la lógica principal.
Adapter	Estructural	EmailNotificationAdapter adapta el servicio externo de correo a la interfaz de notificaciones del sistema.	Desacopla el dominio de proveedores externos y facilita cambiar el canal de envío.
Observer	Comportamiento	TicketObserver reacciona ante cambios de estado o asignación para generar notificaciones.	Permite notificar sin acoplar TicketService con cada canal de comunicación.
Repository	Arquitectónico	TicketRepository y UserRepository aíslan la persistencia de datos.	Mejora pruebas, mantenibilidad y sustitución de la base de datos.

10. Flujo principal del incidente

1. El usuario registra un ticket con descripción, categoría y evidencias.
2. El sistema asigna estado Nuevo y genera notificación de creación.
3. El coordinador revisa prioridad, SLA y disponibilidad del equipo.
4. El ticket se asigna a un técnico, quien registra diagnóstico y avances.
5. Si el caso requiere mayor especialización, se escala a otro nivel.
6. Cuando existe solución, el técnico cambia el estado a Resuelto.
7. El usuario valida la solución o solicita reapertura.
8. El coordinador o técnico cierra el ticket y queda el historial completo.

11. GitHub y flujo Gitflow

El repositorio de GitHub debe contener el documento, diagramas UML, evidencias de avance y un README con la explicación del diseño. El enlace obligatorio para la entrega es: <https://github.com/USUARIO/REPOSITORIO-HELPDESK>

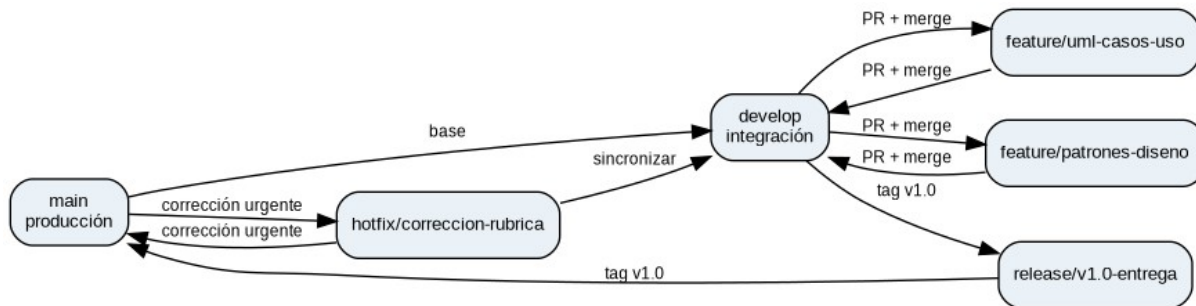


Figura 3. Ramas recomendadas bajo Gitflow.

Rama	Uso	Ejemplo de commits
main	Versión estable final entregada.	v1.0 entrega actividad helpdesk
develop	Integración de avances aprobados.	merge feature/uml-casos-uso
feature/uml-casos-uso	Diseño del diagrama de casos de uso.	agrega actores y casos de uso
feature/modelo-clases	Diseño del diagrama de clases.	define entidades ticket usuario categoria
feature/patrones-diseno	Documentación de patrones aplicados.	documenta factory adapter observer
release/v1.0-entrega	Preparación final del PDF.	ajusta formato y evidencia final

12. Estructura sugerida del repositorio

```
helpdesk-architecture/
├── README.md
├── docs/
│   └── actividad-helpdesk.pdf
├── uml/
│   ├── casos-uso.png
│   ├── clases.png
│   └── gitflow.png
├── evidencias/
│   └── capturas-commits.png
└── src/ (opcional si se desea prototipo)
```

13. Conclusiones

El diseño propuesto organiza el Sistema Help Desk en capas y separa responsabilidades mediante entidades de dominio, servicios de aplicación e infraestructura. Los diagramas UML permiten visualizar actores, casos de uso y clases principales. La incorporación de patrones como Factory Method, Adapter y Observer mejora la extensibilidad y reduce el acoplamiento. Finalmente, Gitflow aporta trazabilidad profesional al proceso de diseño y facilita la entrega controlada de versiones en GitHub.

14. Checklist de entrega

- PDF final generado.
- Enlace del repositorio de GitHub reemplazado por el enlace real.
- Diagramas UML incluidos.
- Mínimo tres patrones de diseño explicados.
- Evidencia de Gitflow en el repositorio.
- Archivo subido antes del cierre: 16 de mayo de 2026, 23:59.